

# Building a Step-Level Process Reward Model

tsushanth · t.sushanth@gmail.com, Jayaprakash Sundararaj

September 2026

## [1. Executive Summary](#)

### [1.1. Quick Stats](#)

## [2. Problem Statement](#)

## [3. Data](#)

### [3.1. Source files and provenance](#)

### [3.2. From solutions to steps](#)

### [3.3. Planting mistakes](#)

### [3.4. Labels](#)

## [4. Architecture](#)

## [5. Training](#)

## [6. Integration into GRPO](#)

### [6.1. A caution about the minimum](#)

## [7. Results](#)

### [7.1. Reward sanity check](#)

### [7.2. What this does not show](#)

## [8. Code Structure & Reproducibility](#)

## [9. Appendix](#)

### [9.1. Discontinued Ideas](#)

### [9.2. Glossary](#)

## 1. Executive Summary

**Headline result:** A working step-level process reward model (PRM) was built on a Qwen2.5-0.5B backbone and trained on the 742 unique teacher solutions with planted mistakes. On 111 questions held out from training, it scores **0.954 AUROC** at telling a correct chain from a corrupted one, with **93.1%** step-level accuracy. It is wired into GRPO as `--reward prm`. This result is in-distribution: the test mistakes are made by the same recipe as the training mistakes.

**In plain terms:** The existing training run only checks whether a model's final number is right. A process reward model marks each line of working instead, so training can reward good reasoning and point at the line that went wrong. This report covers building that marker and connecting it to training. Whether it can be trusted on the model's real mistakes is the subject of the next two reports.

### Findings:

- The training file holds 742 unique GSM8K train questions, each with a teacher solution checked against the reference answer (§3.1). The four data files add up to 1,535 lines only because the merged file repeats the other three.
- The solutions are terse: on average 46 words and 4.1 sentences, so each solution offers only a handful of steps to grade.
- Negative examples were made by corrupting one step of a correct solution in one of three ways (change the result, swap in a wrong number, flip an operator). Held-out AUROC is 0.956, 0.947 and 0.957 for the three types respectively.
- The chain score is the lowest step probability. That rule has a built-in side effect: more steps means more chances to score low (§6).
- The reward hook is opt-in. With the default `--reward outcome`, the reward list is identical to the original.
- Training took 1,116 seconds on a laptop CPU by training only the top 8 layers of the model.
- NOTE: A 0.954 score on planted mistakes says little about real mistakes. Reports 2 and 3 show real-mistake performance is 0.70 to 0.80.
- NOTE: No GRPO run with this reward has been made yet.

## 1.1. Quick Stats

| Metric                                         | Value                                                             |
|------------------------------------------------|-------------------------------------------------------------------|
| Training file (unique questions)               | 742                                                               |
| Source                                         | GSM8K train; teachers DeepSeek R1-0528 and V3.2, answers verified |
| Training / held-out split (by question)        | 631 / 111 questions                                               |
| Training chains (positives + planted mistakes) | 1,839                                                             |
| Held-out chains                                | 325 (111 correct, 214 corrupted)                                  |
| Backbone                                       | Qwen2.5-0.5B-Instruct + linear head                               |
| Trainable layers                               | top 8 of 24, plus the final norm and head                         |
| Training steps                                 | 459 (2 epochs, batch 8)                                           |
| Training time                                  | 1,116 s on CPU                                                    |
| Held-out step accuracy                         | 0.931                                                             |

| Metric               | Value |
|----------------------|-------|
| Held-out chain AUROC | 0.954 |

## 2. Problem Statement

The reasoning pipeline (distill, prepare data, SFT, merge, GRPO, merge, evaluate) trains its reinforcement-learning stage with an **outcome** reward: 1 point if the final number matches the reference, plus a format bonus. Earlier work showed SFT was the only lever that moved accuracy; tripling the data and reshaping the rewards left GSM8K at 0.77 (base), 0.82 (SFT) and 0.79 (GRPO) on 100 questions.

The step-level reward the project brief asked for was never built. The task here: build a small PRM, train it on the existing verified traces plus negatives that do not exist yet, and make it usable as the GRPO reward in place of exact-match. Constraints: a 16 GB laptop for local testing (CPU only, since the GPU backend crashes with the number format used), and rented GPUs only when needed.

## 3. Data

### 3.1. Source files and provenance

The training set is `reasoning_sft.jsonl`: **742 unique GSM8K train questions**, each with a teacher-written step-by-step solution whose final answer matched the reference answer (solutions that did not match were discarded). It is the merged, de-duplicated output of `prep_data.py`, built from the distillation batches below.

Two teacher models wrote the solutions: DeepSeek R1-0528 for the first batch of about 200 problems, then DeepSeek V3.2 for the bulk, after a three-question trial found V3.2 about twice as fast (16.4 s against 34.9 s per call) with the same 3-of-3 matches. The teacher is not recorded on each row, so which model wrote which row cannot be recovered.

| File                                   | Rows | Role                                                                                        |
|----------------------------------------|------|---------------------------------------------------------------------------------------------|
| <code>reasoning_sft.jsonl</code>       | 742  | The merged training file (unique questions)                                                 |
| <code>reasoning_sft_more.jsonl</code>  | 203  | First V3.2 distillation batch                                                               |
| <code>reasoning_sft_more2.jsonl</code> | 574  | Second batch                                                                                |
| <code>reasoning_sft_more3.jsonl</code> | 16   | Third batch; the model-API key was rate-limited, so only 16 of 600 requested rows came back |

Checked directly against the files: batches 1 and 2 share 83 questions; the three batches together hold 710 unique questions, all of which are in the merged file; 32 questions appear only in the merged file (probably from the earlier, smaller trace set; not verified). The four line counts add up to 1,535, but that number double counts because the merged file repeats the batches.

Duplicates are removed by question text. The project history records the same trap once before: a dataset described as “438 rows” held only 235 unique questions. Every count in this report is after de-duplication.

### 3.2. From solutions to steps

A solution is split into steps at line breaks and at sentence boundaries (a full stop, question mark or exclamation mark followed by a capital letter, digit, dollar sign or bracket). Decimals such as \$1.75 are not split. Fragments under three characters are dropped. The same splitter is used in training and, later, inside the GRPO reward, so the model sees steps cut the same way in both places.

Example, split into two steps:

Perimeter formula:  $2(L + W) = 30$ . | Given  $L = 2W$ , substitute:  $2(2W + W) = 30 \rightarrow 2(3W) = 30 \rightarrow 6W = 30 \rightarrow W = 5$ .

### 3.3. Planting mistakes

Each question yields one correct chain and up to two corrupted chains. A corrupted chain changes exactly one step, chosen at random, using one of three mutations:

| Mutation                    | What it does                                                                                                                      | Example                                          |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <code>perturb_result</code> | Changes the last number in the step to a nearby wrong value (off by a small amount, scaled by 2, 0.5 or 10, or one digit shifted) | “ $W = 5$ .” becomes “ $W = 4$ .”                |
| <code>swap_operand</code>   | Replaces a non-final number in the step with a different quantity from the problem                                                | “ $2(L + W) = 30$ ” becomes “ $30(L + W) = 30$ ” |
| <code>flip_op</code>        | Turns $+$ into $-$ , $*$ into $/$ , and similar                                                                                   | “ $2 + 3$ ” becomes “ $2 - 3$ ”                  |

On the first 200 questions this produced 580 chains: 200 correct, 177 `perturb_result`, 108 `swap_operand`, 95 `flip_op`. Not every step admits every mutation, so the mix is uneven.

### 3.4. Labels

Steps before the corrupted one are labelled 1 (correct). The corrupted step is labelled 0. Steps after it are **masked** (ignored in the loss), because once a chain has gone wrong, whether later steps are “correct” is ambiguous. This is the usual convention for step-level reward models.

The split into training and held-out sets is by **question**, not by chain, so the model is never tested on a question it saw in another form.

## 4. Architecture

The model is a Qwen2.5-0.5B backbone with a single linear layer on top. The input is the question followed by the steps, each step ending in a separator token (`<|file_sep|>`, a special token in the Qwen vocabulary that normal text does not use). The linear layer reads the model's internal state at each separator and outputs one number, turned into a probability that the step is correct.

Because the backbone is **causal** (each position sees only what came before), the score at step  $k$  depends on the question and steps 1 to  $k$  only. It cannot peek at later steps.

```
Problem: <question>
Solution:
<step 1> <sep>    -> P(step 1 correct)
<step 2> <sep>    -> P(step 2 correct | step 1)
<step 3> <sep>    -> P(step 3 correct | steps 1-2)
```

| Setting                          | Value                                                                 |
|----------------------------------|-----------------------------------------------------------------------|
| Backbone                         | Qwen/Qwen2.5-0.5B-Instruct via AutoModel (no language-model head)     |
| Head                             | Linear(hidden_size, 1)                                                |
| Maximum length                   | 512 tokens; the question is clipped to 256 tokens so steps always fit |
| Chain score                      | minimum step probability (a mean option exists)                       |
| Empty chain (no parseable steps) | scores 0                                                              |

## 5. Training

Training used a plain PyTorch loop rather than a library trainer.

| Setting        | Value                                            |
|----------------|--------------------------------------------------|
| Layers trained | top 8 of 24 layers, the final norm, and the head |
| Optimiser      | AdamW, weight decay 0.01                         |
| Learning rate  | 5e-5 for the backbone layers, 1e-3 for the head  |
| Schedule       | linear decay to 5% of the starting rate          |

| Setting                | Value                                                                 |
|------------------------|-----------------------------------------------------------------------|
| Batch / epochs / steps | 8 chains / 2 / 459                                                    |
| Loss                   | binary cross-entropy on labelled steps only, gradient clipping at 1.0 |
| Numeric precision      | 32-bit floating point on CPU                                          |
| Speed                  | about 3 seconds per step; 1,116 seconds in total                      |

**Why only the top 8 layers.** The LoRA library used elsewhere in the project was not installed locally, and training all layers on a laptop CPU would have been slow. Freezing the lower 16 layers cut the backward pass to a fraction of its cost. Report 3 documents what happens when this is changed on a GPU: full training at the same learning rate failed.

**Smoke test.** Before the full run, a 60-question, 18-step run confirmed the code end to end. Its held-out AUROC was 0.506, which is chance and is what a model trained for 56 seconds on 51 questions should score.

## 6. Integration into GRPO

The reward function `make_prm_reward` in `rewards.py` loads the PRM once, on first use, then scores each completion the trainer produces: it extracts the text between `<reasoning>` tags, splits it into steps, scores every step, and returns the chain score. A completion with no parseable reasoning scores 0. Scoring is batched (16 chains at a time), in bf16 on a GPU or fp32 on CPU.

The trainer library forwards extra dataset columns as arguments, so the reward receives the original `question` alongside each completion.

`train_grpo.py` gained three options:

| Option                                  | Meaning                                       |
|-----------------------------------------|-----------------------------------------------|
| <code>--reward outcome</code> (default) | Original rewards: exact-match, format, length |
| <code>--reward prm</code>               | PRM replaces exact-match: PRM, format, length |
| <code>--reward prm+outcome</code>       | Both: PRM, exact-match, format, length        |
| <code>--prm</code>                      | Path to the trained PRM directory             |
| <code>--prm-aggregate min\lmean</code>  | How step scores combine into a chain score    |

The existing rewards are unchanged: format gives +1.0 for a reasoning block and an answer, −0.5 for an answer only, −2.0 for neither; length gives up to 1.0 for completions near 400 characters. GRPO defaults are learning rate 1e-6, KL weight 0.04, 8 attempts per question.

`eval_judge.py` also gained average-output-token tracking (overall and on correct answers), because the claim under test is that a step-level reward gives shorter chains at equal accuracy.

## 6.1. A caution about the minimum

Taking the lowest step probability makes the score fall as steps are added: each extra step is another chance to be scored low. A model trained against this reward could learn to write fewer steps without reasoning any better. Any “shorter chains” result therefore needs an outcome-reward control run alongside it.

## 7. Results

On 111 held-out questions (325 chains: 111 correct, 214 corrupted):

| Metric                                      | Value |
|---------------------------------------------|-------|
| Step accuracy                               | 0.931 |
| Chain AUROC (weakest-step score)            | 0.954 |
| AUROC on <code>perturb_result</code> chains | 0.956 |
| AUROC on <code>swap_operand</code> chains   | 0.947 |
| AUROC on <code>flip_op</code> chains        | 0.957 |

### 7.1. Reward sanity check

Six questions were each scored three ways through the actual reward function: the correct completion, a copy with a wrong result inserted, and untagged text.

| Case                  | Score range                                             |
|-----------------------|---------------------------------------------------------|
| Correct completion    | 0.79 to 0.98                                            |
| Wrong result inserted | 0.00 to 0.89 (0.000, 0.000, 0.516, 0.611, 0.893, 0.040) |
| Untagged text         | 0.000 in all six                                        |

One of the six scored the corrupted copy (0.893) about the same as the correct one (0.862). The wrong result is only inserted when the answer string appears in the reasoning, so that case may not have been altered; this was not checked. Treat the check as a plumbing test, not evidence of accuracy.

### 7.2. What this does not show

The held-out mistakes come from the same three-mutation recipe as the training mistakes, so the score measures how well the model learned that recipe. A model could reach 0.95 by learning “does the arithmetic add up” and still miss a wrong setup or a misread question. Report 2 tests this directly.

# 8. Code Structure & Reproducibility

Branch `reasoning/prm`, created from `reasoning/ida-loop` (commit 1ad415f) in a separate git worktree so other work was not disturbed. Pushed commit: 79952a9.

```
training/reasoning/
prm.py           split_steps, corrupt, build_examples, PRM model, scoring
train_prm.py     training loop, held-out evaluation (AUROC by corruption type)
rewards.py       make_prm_reward (new); original rewards unchanged
train_grpo.py    --reward, --prm, --prm-aggregate (new)
eval_judge.py    average output tokens (new)
```

To reproduce the PRM (needs the untracked data files):

```
python -m training.reasoning.train_prm --train-layers 8 --epochs 2 --output outputs/prm
```

The trained weights (about 2 GB) are gitignored. Only the CPU-trained version from this report exists locally; retraining on a GPU takes a few minutes.

# 9. Appendix

## 9.1. Discontinued Ideas

**LoRA for the PRM.** The LoRA library was not installed on the laptop, and adding it was not worth the setup for a 0.5B model. Freezing lower layers achieved the same speed-up with no new dependency.

**GPU backend on the laptop.** The project notes say the Apple GPU backend crashes with the number format and memory settings used in this pipeline. All local runs were CPU-only.

**Output-path bug.** The first smoke run wrote its output next to the data files, which live in a different checkout than the code. The stray output was deleted and the path fixed to always resolve beside the code.

## 9.2. Glossary

| Term                       | Plain-language meaning                                                                                                             |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| PRM (process reward model) | A small model that reads a question and the working so far and outputs, after each step, the probability that the step is correct. |
| Outcome vs process reward  | An outcome reward marks only the final answer. A process reward marks every line of working.                                       |

| Term                            | Plain-language meaning                                                                                                                                                                            |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Step                            | One line or sentence of working. Solutions are split into steps automatically at line and sentence breaks.                                                                                        |
| Corruption (synthetic negative) | A fake mistake planted in a correct solution (a changed result, a swapped number, a flipped operator) to teach the PRM what bad working looks like.                                               |
| AUROC                           | How well a scorer separates right from wrong solutions: the chance it gives a randomly chosen right solution a higher score than a randomly chosen wrong one. 0.5 is a coin flip, 1.0 is perfect. |
| Exact-match reward              | 1 point if the final number equals the correct answer, otherwise 0. It only looks at the end, so it is an outcome reward.                                                                         |
| GRPO                            | The reinforcement-learning recipe used in this pipeline. For each question the model writes several attempts, they are scored, and it is nudged toward the better-scoring ones.                   |
| Token                           | The unit a model reads and writes, roughly three-quarters of a word. Average output tokens is our measure of answer length.                                                                       |
| Fine-tuning                     | Further training of an existing model on a specific job instead of training from scratch.                                                                                                         |
| SFT                             | Supervised fine-tuning: showing the model worked examples (question, then written solution) so it learns to imitate them.                                                                         |